INSTITUTO TECNOLÓGICO DE TIJUANA

Carrera:

Nombre de estudiante:

Soto Carrillo Misael 21210431

Bejarano Duque Antoine 21211918

Noel Andre Mendez Adame 21211993

Viera Pérez Gerardo 21212735

Materia:

PATRONES DE DISEÑO DE SOFTWARE

Unidad y tema:

Unidad #2 Actividad de evidencia Equipo_U2 Abstract Factory

Periodo:

Ene – Ago 2026

Grupo:

SC9A

Profesor(a):

MARIBEL GUERRERO LUIS

Horario:

Lunes a viernes de 15:00 - 16:00

Fecha: Tijuana, Baja California a 17 de marzo 2026

Índice

Introducción 2
Qué es el patrón utilizado 3
Cómo se evita el usar tanto NEW al usar este patrón. 3
Cómo agregar una nueva cocina 3
Código en consola y utilización 4
Ejecución del programa 10
  Restaurante de comida china 10
  Restaurante de comida Japonesa 11
  Restaurante de comida mexicana 11
  Restaurante de comida italiana 12
  Restaurante de comida griega 12
Diagrama UML 13
  Parte principal del código 14
  Fabricas creadas 14
    Cocina china 14
    Cocina japonesa 15
    Cocina mexicana 15
    Cocina italiana 16
    Cocina Griega 16
Conclusión 17

Introducción

Abstract Factory es un patrón de diseño creacional que se utiliza en la programación orientada a objetos, y permite crear familias de objetos vinculados sin tener que basarse en sus clases específicas. Este modelo separa la lógica de creación de objetos del resto del sistema, lo que mejora la organización del código, simplifica su mantenimiento y posibilita modificaciones o ampliaciones sin impactar en otras partes del programa. Asimismo, asegura que los objetos creados dentro de una misma familia sean consistentes.

Qué es el patrón utilizado

El modelo de diseño que se emplea en el diagrama es el patrón de diseño Fábrica abstracta. Aplicado en nuestro software este patrón como se muestra en el código y las imágenes nos permite crear múltiples fabricas abstractas permitiendo una opción más concreta simple para el usuario. Lo cual nos permite cambiar el menú por completo solo sustituyendo la clase utilizada sin necesidad de recibir todo el código.

Cómo se evita el usar tanto NEW al usar este patrón.

Con este patrón la creación de los objetos se delega a una fábrica dejando que el cliente como se muestra en este apartado del código

```
CocinaFactory fabrica = new CocinaJaponesaFactory();

PlatoFuerte plato = fabrica.crearPlatoFuerte();
Bebida bebida = fabrica.crearBebida();
Postre postre = fabrica.crearPostre();
```

Como lo muestra el ejemplo no se crean directamente los objetos sino que la fábrica se encarga de ellos

Cómo agregar una nueva cocina

En nuestro caso de agregar una nueva cocina es necesario como ejemplo

- Crear los nuevos productos plato fuerte, bebida y postre.
- Crear una nueva fábrica llamada CocinaFrancesaFactory.
- Hacer que esa fábrica genere los nuevos productos.

Para agregar una nueva cocina utilizamos la como ejemplo

```
CocinaFactory fabrica = new CocinaFrancesaFactory();
```

Y en total automatización el sistema estaría sirviendo comida francesa.

Código en consola y utilización

Clase Program

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Threading.Tasks;
using System.Windows.Forms;

namespace Abstract_Factory_Restaurant
{
    internal static class Program
    {
        /// <summary>
        /// The main entry point for the application.
        /// </summary>
        [STAThread]
        static void Main()
        {
            Application.EnableVisualStyles();
            Application.SetCompatibleTextRenderingDefault(false);
            Application.Run(new Form1());
        }
    }
}
```

Clase RestauranteFactory

```
namespace Abstract_Factory_Restaurant
{
    public abstract class RestauranteFactory
    {
        public abstract Platillo CrearPlatillo();
        public abstract Bebida CrearBebida();
        public abstract Postre CrearPostre();
    }

}
```

Clase Cliente

```
namespace Abstract_Factory_Restaurant
{
    public class Cliente
    {
        private RestauranteFactory fabrica;

        public Cliente(RestauranteFactory fabrica)
        {
            this.fabrica = fabrica;
        }

        public string ObtenerPlatillo()
        {
            return fabrica.CrearPlatillo().Mostrar();
        }
```

```
        public string ObtenerBebida()
        {
            return fabrica.CrearBebida().Mostrar();
        }

        public string ObtenerPostre()
        {
            return fabrica.CrearPostre().Mostrar();
        }
    }

}
```

Clase Bebida

```
namespace Abstract_Factory_Restaurant
{
    public abstract class Bebida
    {
        public abstract string Mostrar();
    }
}
```

Clase Platillo

```
namespace Abstract_Factory_Restaurant
{
    public abstract class Platillo
    {
        public abstract string Mostrar();
    }
}
```

Clase Postre

```
namespace Abstract_Factory_Restaurant
{
    public abstract class Postre
    {
        public abstract string Mostrar();
    }

}
```

Clase RestauranteChinoFactory

```
namespace Abstract_Factory_Restaurant
{
    public class PlatilloChina : Platillo
    {
        public override string Mostrar()
        {
            return "Arroz frito";
        }
    }

    public class BebidaChina : Bebida
    {
        public override string Mostrar()
```

```
        {
            return "Té verde";
        }
    }

    public class PostreChina : Postre
    {
        public override string Mostrar()
        {
            return "Pastel de luna";
        }
    }

    public class RestauranteChinoFactory : RestauranteFactory
    {
        public override Platillo CrearPlatillo()
        {
            return new PlatilloChina();
        }

        public override Bebida CrearBebida()
        {
            return new BebidaChina();
        }

        public override Postre CrearPostre()
        {
            return new PostreChina();
        }
    }
}
```

Clase RestauranteGriegoFactory

```
namespace Abstract_Factory_Restaurant
{
    public class PlatilloGriego : Platillo
    {
        public override string Mostrar()
        {
            return "Gyro";
        }
    }

    public class BebidaGriega : Bebida
    {
        public override string Mostrar()
        {
            return "Café griego";
        }
    }

    public class PostreGriego : Postre
    {
        public override string Mostrar()
        {
            return "Baklava";
        }
```

```
    }

    public class RestauranteGriegoFactory : RestauranteFactory
    {
        public override Platillo CrearPlatillo()
        {
            return new PlatilloGriego();
        }

        public override Bebida CrearBebida()
        {
            return new BebidaGriega();
        }

        public override Postre CrearPostre()
        {
            return new PostreGriego();
        }
    }
}
```

Clases RestauranteItalianoFactory

```
namespace Abstract_Factory_Restaurant
{
    public class PlatilloItaliano : Platillo
    {
        public override string Mostrar()
        {
            return "Pizza";
        }
    }

    public class BebidaItaliana : Bebida
    {
        public override string Mostrar()
        {
            return "Espresso";
        }
    }

    public class PostreItaliano : Postre
    {
        public override string Mostrar()
        {
            return "Tiramisú";
        }
    }

    public class RestauranteItalianoFactory : RestauranteFactory
    {
        public override Platillo CrearPlatillo()
        {
            return new PlatilloItaliano();
        }

        public override Bebida CrearBebida()
        {
            return new BebidaItaliana();
```

```
        }

        public override Postre CrearPostre()
        {
            return new PostreItaliano();
        }
    }
}
```

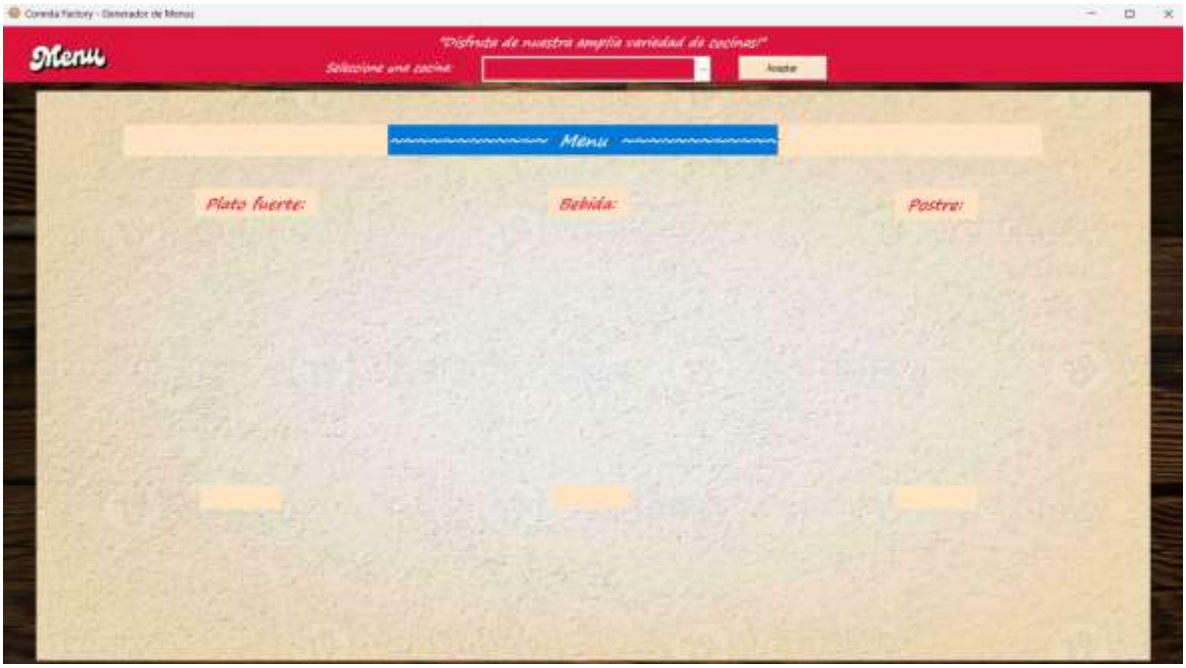
Clases RestauranteJaponesFactory

```
namespace Abstract_Factory_Restaurant
{
    public class PlatilloJapones : Platillo
    {
        public override string Mostrar()
        {
            return "Sushi";
        }
    }

    public class BebidaJaponesa : Bebida
    {
        public override string Mostrar()
        {
            return "Sake";
        }
    }

    public class PostreJapones : Postre
    {
        public override string Mostrar()
        {
            return "Mochi";
        }
    }

    public class RestauranteJaponesFactory : RestauranteFactory
    {
        public override Platillo CrearPlatillo()
        {
            return new PlatilloJapones();
        }

        public override Bebida CrearBebida()
        {
            return new BebidaJaponesa();
        }

        public override Postre CrearPostre()
        {
            return new PostreJapones();
        }
    }
}
```

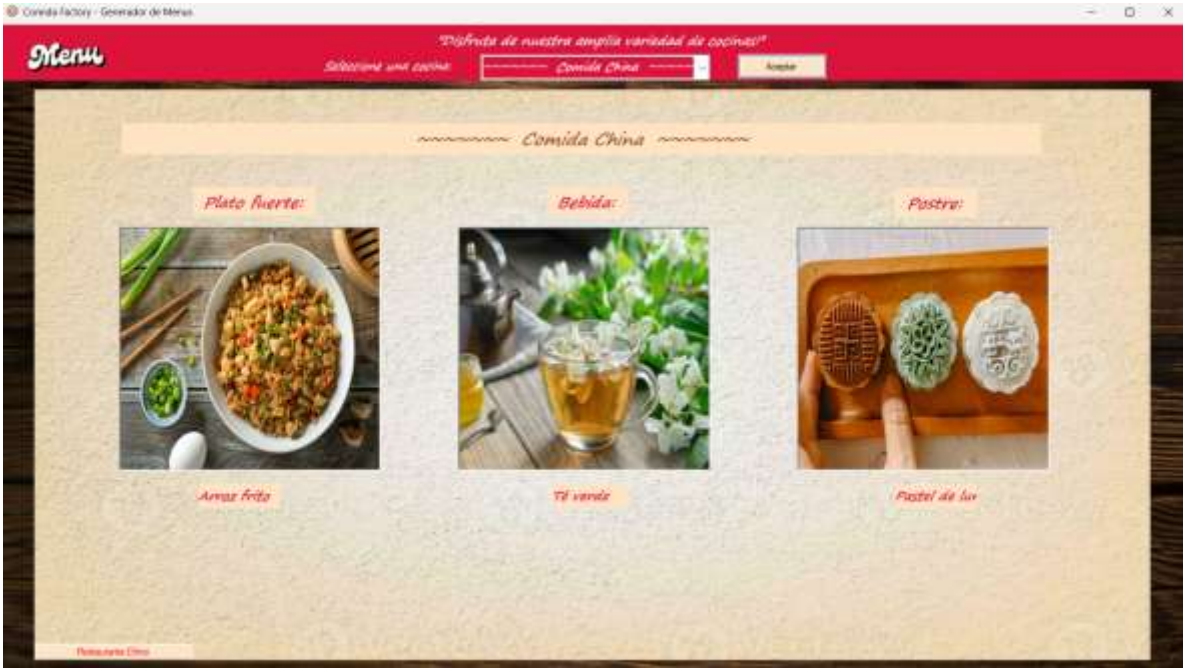
Clases RestauranteMexicanoFactory

```
namespace Abstract_Factory_Restaurant
{
    public class PlatilloMexicano : Platillo
    {
        public override string Mostrar()
        {
            return "Tacos";
        }
    }

    public class BebidaMexicana : Bebida
    {
        public override string Mostrar()
        {
            return "Agua de horchata";
        }
    }

    public class PostreMexicano : Postre
    {
        public override string Mostrar()
        {
            return "Churros";
        }
    }

    public class RestauranteMexicanoFactory : RestauranteFactory
    {
        public override Platillo CrearPlatillo()
        {
            return new PlatilloMexicano();
        }

        public override Bebida CrearBebida()
        {
            return new BebidaMexicana();
        }

        public override Postre CrearPostre()
        {
            return new PostreMexicano();
        }
    }
}
```

Ejecución del programa



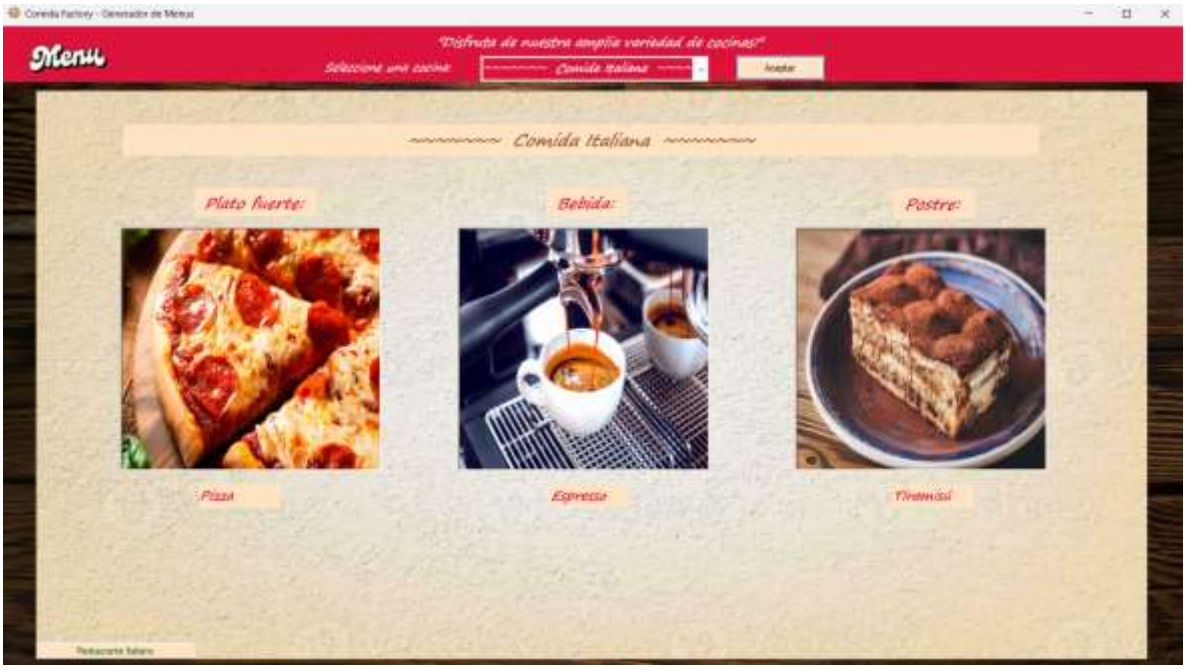
Restaurante de comida china



Restaurante de comida Japonesa

Restaurante de comida mexicana

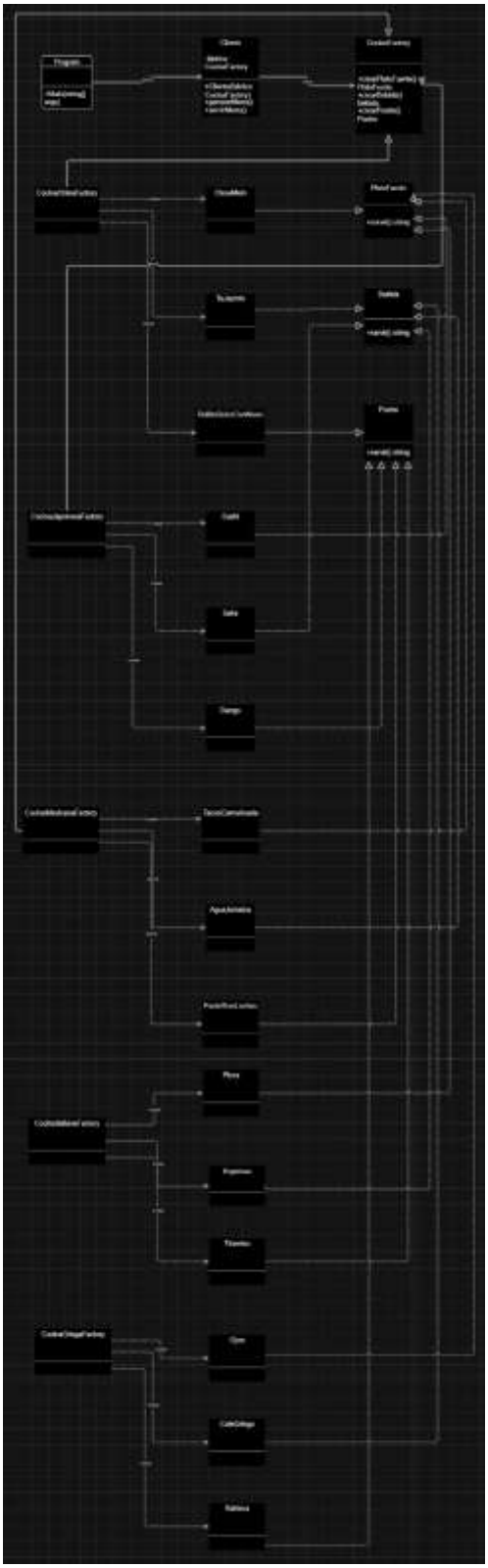
Restaurante de comida italiana



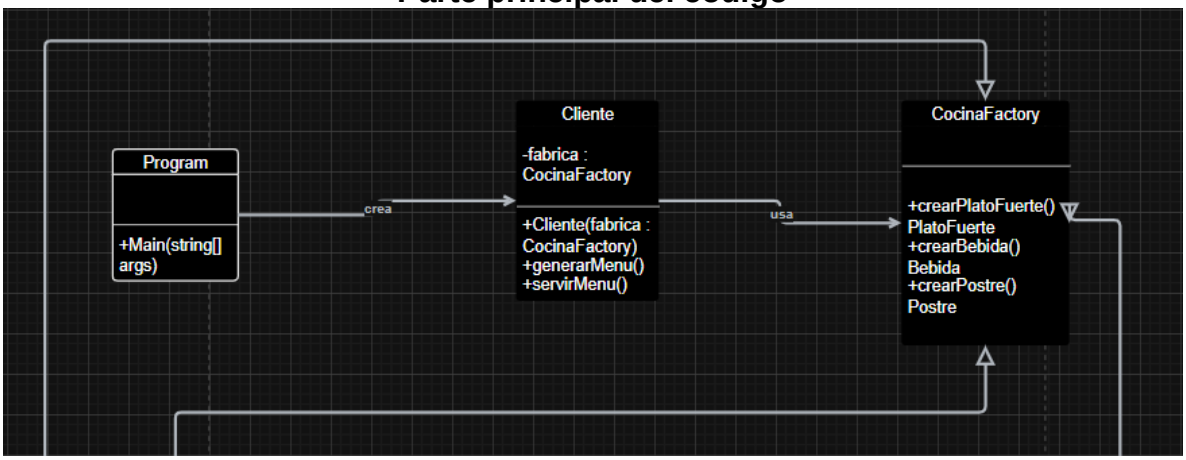
Restaurante de comida griega

Diagrama UML

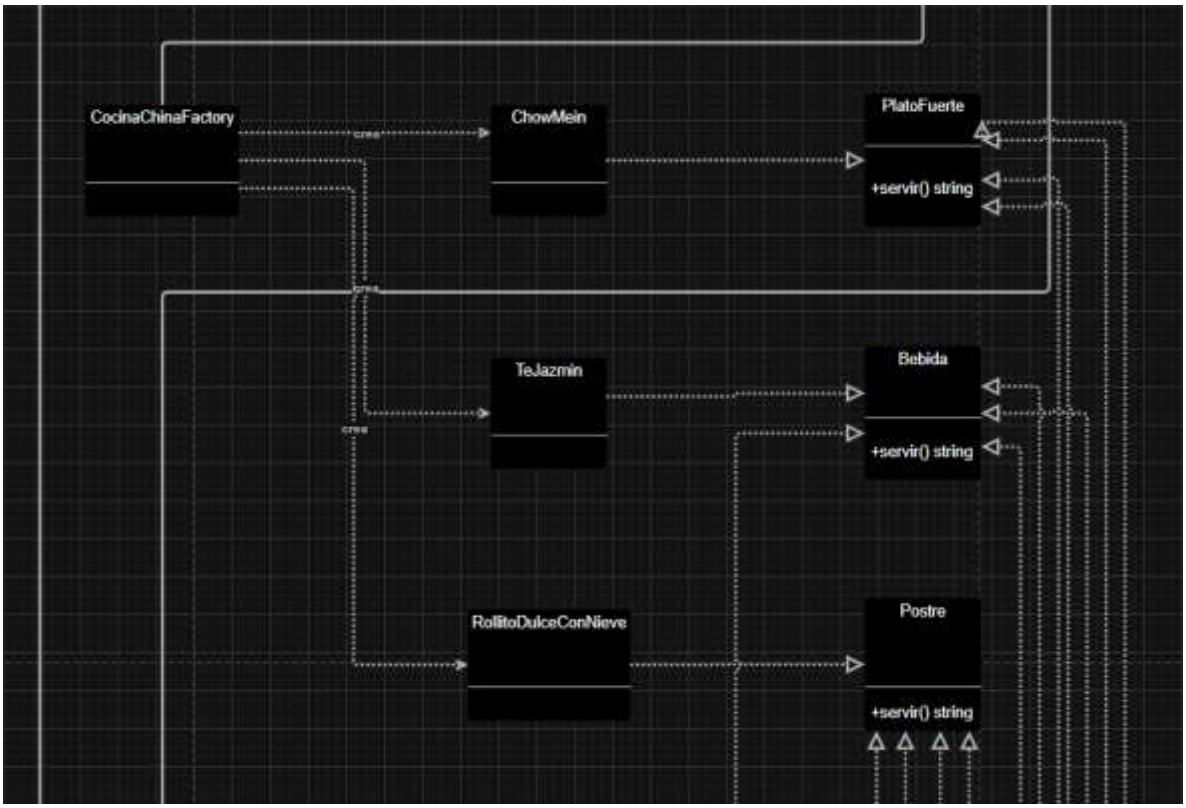
https://drive.google.com/file/d/1EapCzySDI7egWlA2nHmm181G6Z40HerV/view?usp=sharing



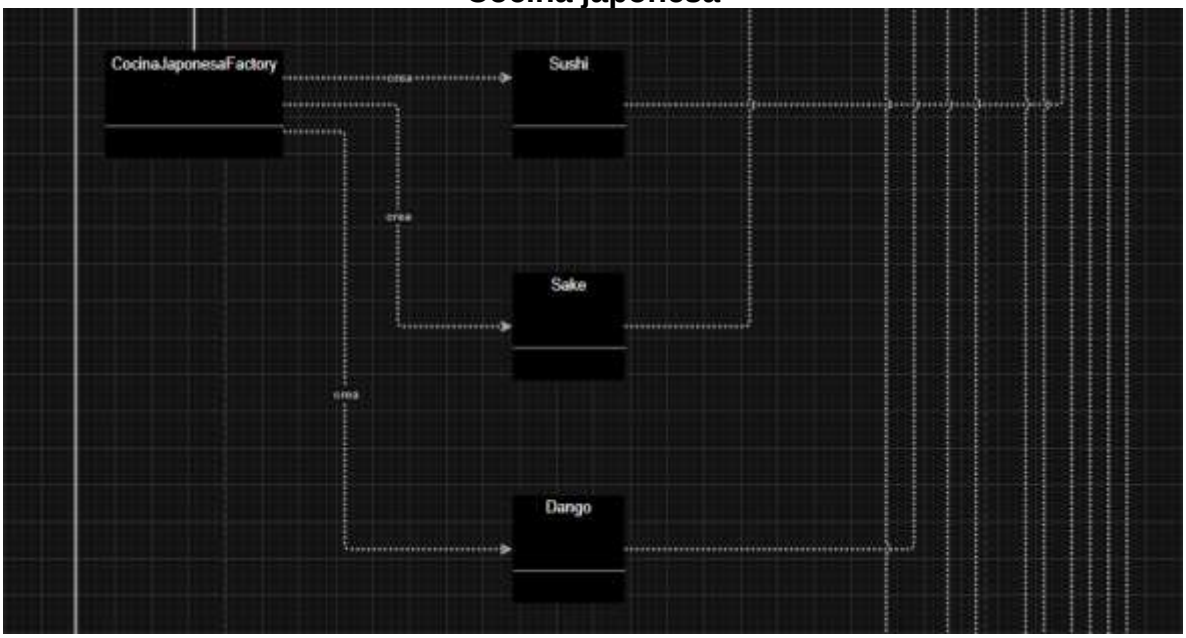
Parte principal del código



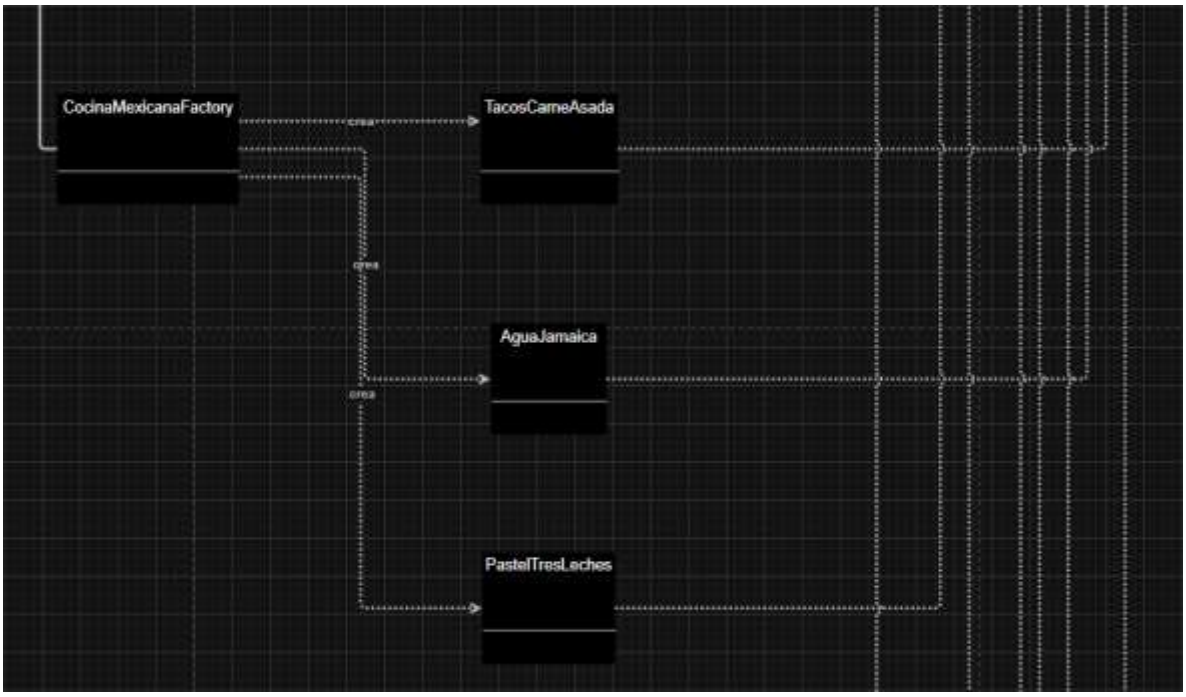
Fabricas creadas
Cocina china



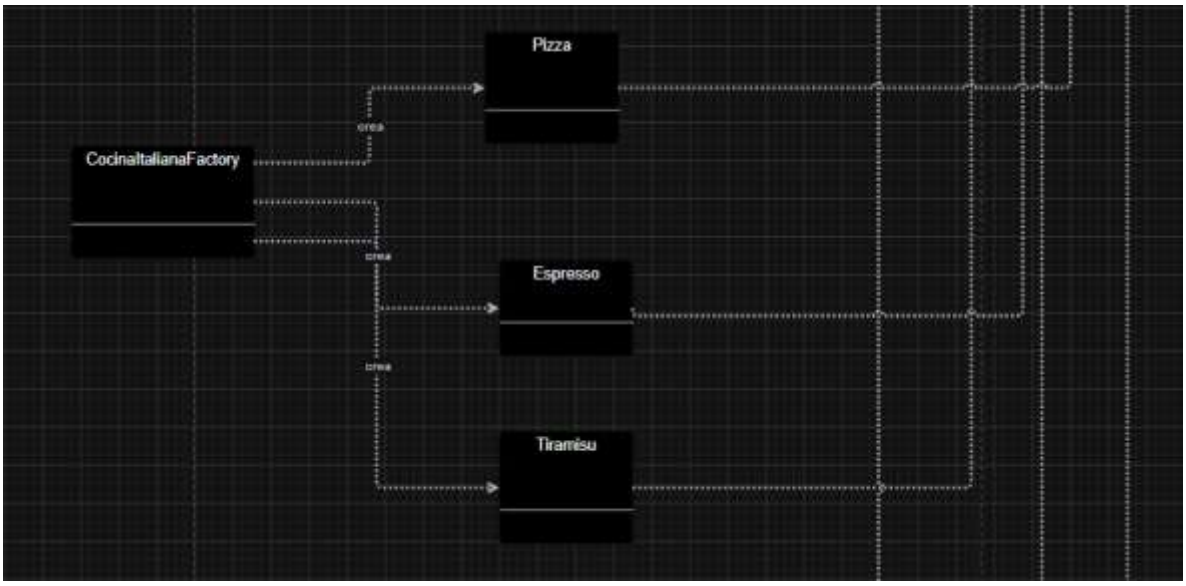
Cocina japonesa



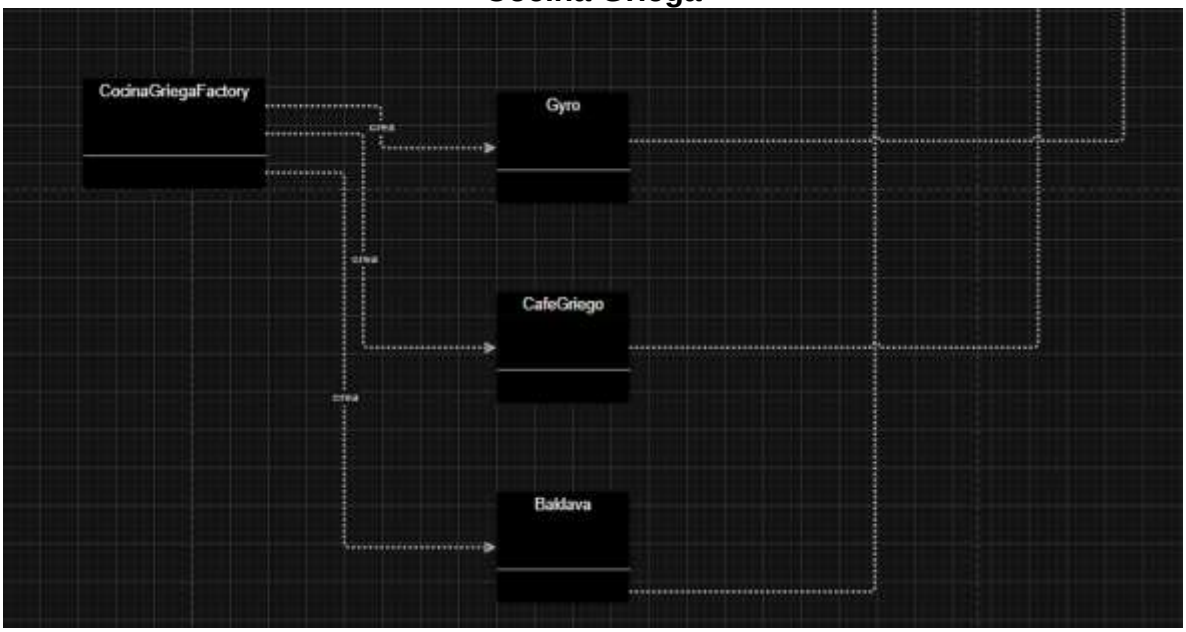
Cocina mexicana



Cocina italiana



Cocina Griega



Conclusión

El patrón Abstract Factory es una herramienta muy útil para el diseño de software porque permite generar familias de objetos relacionados entre sí de manera ordenada y flexible. Es más fácil mantener, actualizar y reutilizar el código a largo plazo si se separa la creación de los objetos de su implementación particular. Este patrón, además, ayuda a que los sistemas sean más escalables, pues permite agregar nuevas variantes de objetos sin que esto afecte el funcionamiento global del software. Por esta razón, se emplea a menudo en proyectos que deben lidiar con diferentes tipos de objetos que tienen atributos similares.